

Assignment #1



- 과 목 명 : 자료구조
- 담 당 교 수 님 : 김무철 교수님
- 학 과 : 소프트웨어학부
- 보 고 서 작 성 자 : 서정민

Data Structure Assignment #1

1-1 : Implementation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5  #include <Windows.h>
6
7  #define _CRT_SECURE_NO_WARNINGS
8
9  LARGE_INTEGER start, end, freq;
10
11 void StartTimer() {
12     QueryPerformanceFrequency(&freq);
13     QueryPerformanceCounter(&start);
14 }
15
16 double EndTimer() {
17     QueryPerformanceCounter(&end);
18     return ((double)(end.QuadPart - start.QuadPart) / (double)freq.QuadPart) * 1000000; // microseconds
19 }
20
```

QueryPerformanceCounter() 함수를 사용하여 실행시간을 측정합니다. StartTimer, EndTimer 함수로 microseconds 단위로 시간을 측정할수 있도록 합니다.

```
23 // dynamic arrays struct
24 typedef struct DynamicArray {
25     int* d_array;
26     int capacity; // 크기
27     int size;     // 저장된 원소 수
28 }DynamicArray;
29
```

DynamicArray 구조체를 정의합니다. d_array 에는 배열, capacity 에는 전체 배열의 크기, size 에는 현재 저장된 원소의 수를 저장합니다. 이 과제에서 구현 중 자료에 담는 data 는 int 로 정하고 구현하였습니다.

```
30 // dynamic_array initialization with a specific size
31 struct DynamicArray* dynamic_array_init(int input_size, int input_capacity) {
32     DynamicArray* D_array = (struct DynamicArray*)malloc(sizeof(struct DynamicArray));
33     if (D_array == NULL) {
34         return NULL;
35     }
36
37     D_array->d_array = (int*)malloc(input_capacity * sizeof(int));
38     if (D_array->d_array == NULL) {
39         free(D_array);
40         return NULL;
41     }
42
43     //0으로 내용물 초기화
44     D_array->capacity = input_capacity;
45     D_array->size = input_size;
46     memset(D_array->d_array, 0, sizeof(int) * (D_array->capacity));
47
48     return D_array;
49 }
50
```

Dynamic_array_init 함수에서 input_size, input_capacity 를 매개변수로 입력받아서 동적으로 할당받은 후 return 합니다.

```
50
51 // 삽입 중 배열 복사를 위한 함수
52 void copy_array(int* arr1, int* arr2, int size) {
53     // arr1에서 arr2로 복사한다. size는 arr1의 크기
54     for (int i = 0; i < size; i++)
55         *(arr2 + i) = *(arr1 + i);
56 }
57
58 // 요소 삽입중 크기조절을 위한 함수
59 void dynamic_array_resize(DynamicArray* D_array, int new_capacity) {
60     int* new_array = (int*)malloc(sizeof(int) * new_capacity);
61     if (new_array == NULL) {
62         return;
63     }
64     for (int i = 0; i < D_array->size; i++) {
65         new_array[i] = D_array->d_array[i];
66     }
67     free(D_array->d_array);
68     D_array->d_array = new_array;
69     D_array->capacity = new_capacity;
70 }
```

Dynamic array 에 새로운 요소 삽입중 필요한 배열 복사와 크기 조절을 위한 함수를 정의합니다. Copy_array 함수에서는 arr1, arr2, arr1 의 크기를 입력받고 arr1 에서 arr2 로 element 를 복사합니다. Dynamic_array_resize 함수에서는 DynamicArray 구조체와 new_capacity 를 입력받아서 새로 입력받은 capacity 로 새로 배열을 동적으로 할당받고, 요소를 복사한 뒤 원래 배열의 메모리는 해제합니다.

```
72 // inserting an element(back)
73 struct DynamicArray* dynamic_array_insert_back(DynamicArray* D_array, int insert_element) {
74     // 만약 새로 저장시 capacity를 넘어갈경우, 다시 할당해줘야 한다
75     if (D_array->size == D_array->capacity) {
76         dynamic_array_resize(D_array, D_array->capacity * 2);
77     }
78
79     D_array->d_array[D_array->size] = insert_element;
80     D_array->size++;
81     return D_array;
82 }
```

dynamic_array_insert_back 함수에서는 Dynamic_array 의 뒤에 element 를 삽입합니다. 만약 새로 element 를 삽입시 capacity 를 넘어갈경우, dynamic_array_resize 함수로 capacity 를 2 배로 늘리고 삽입합니다. 배열의 맨 끝 자리에 insert_element 를 삽입하고, size 를 1 늘려서 구조체를 반환합니다.

```

86 // inserting an element(front)
87 struct DynamicArray* dynamic_array_insert_front(DynamicArray* D_array, int insert_element) {
88     // 만약 새로 저장시 capacity를 넘어갈경우, 다시 할당해줘야 한다
89     if (D_array->size == D_array->capacity) {
90         dynamic_array_resize(D_array, D_array->capacity * 2);
91     }
92
93     // 맨 앞에 삽입한다. 전체를 한칸씩 뒤로 밀고 맨 앞자리에 삽입
94     for (int i = D_array->size; i > 0; i--) {
95         D_array->d_array[i] = D_array->d_array[i - 1];
96     }
97     D_array->d_array[0] = insert_element;
98     D_array->size++;
99
100     return D_array;
101 }

```

dynamic_array_insert_front 함수에서는 DynamicArray 구조체와 insert_element 를 매개변수로 입력받아서 맨 앞에 삽입합니다. 만약 새로 삽입할 때 capacity 를 넘어갈 경우 dynamic_array_resize 함수로 capacity 를 2 배로 늘려줍니다. 배열의 전체를 한칸씩 뒤로 옮겨준 뒤에 0 번 index 자리에 insert_element 를 삽입합니다. Size 를 1 늘려주고 DynamicArray 구조체를 반환합니다.

```

103 // inserting an element(specific)
104 struct DynamicArray* dynamic_array_insert_specific(DynamicArray* D_array, int insert_element, int insert_index) {
105     // 만약 새로 저장시 capacity를 넘어갈경우, 다시 할당해줘야 한다
106     if (insert_index > D_array->size) {
107         printf("index out!!\n");
108         return D_array;
109     }
110     if (insert_index == 0) {
111         D_array = dynamic_array_insert_front(D_array, insert_element);
112         //printf("ffff\n"); 0번 인덱스에 insert
113         return D_array;
114     }
115     else if (insert_index == D_array->size) {
116         D_array = dynamic_array_insert_back(D_array, insert_element);
117         //printf("bbbb\n"); 마지막 인덱스에 insert
118         return D_array;
119     }
120     else {
121         if (D_array->size == D_array->capacity) {
122             dynamic_array_resize(D_array, D_array->capacity * 2);
123         }
124
125         // insert_element를 insert_index 자리에 넣고 나머지를 뒤로 + 1(sizeof(int))
126         // 만약 맨 마지막것 + 시 capacity넘어갈경우 다시 할당
127         for (int i = D_array->size; i > insert_index; i--) {
128             D_array->d_array[i] = D_array->d_array[i - 1];
129         }
130         D_array->d_array[insert_index] = insert_element;
131         D_array->size++;
132         return D_array;
133     }
134 }

```

dynamic_array_insert_specific 함수에서는 DynamicArray 구조체와 insert_element, insert index 를 입력받습니다. Insert_index 가 인덱스의 범위를 벗어날경우 오류 메시지를 출력합니다. Insert_index 가 0 번이라면 dynamic_array_insert_front 를 호출하여 삽입후 반환합니다. Insert_index 가 배열의 마지막+1 이라면 dynamic_array_insert_back 함수를 호출여 삽입후

반환합니다. 인덱스가 0 과 마지막+1 이 아닐 경우 d_array 를 insert_index 부터 맨 마지막까지를 한 칸 뒤로 옮겨주고, d_array[insert_index] 자리에 insert_element 를 삽입합니다. 그리고 구조체를 반환합니다.

```
136 // deleting an element(front)
137 struct DynamicArray* dynamic_array_delete_front(DynamicArray* D_array) {
138     if (D_array->size == 0) {
139         return D_array; // 삭제할 요소가 없는 경우 그대로 반환
140     }
141
142     for (int i = 0; i < D_array->size; i++) {
143         D_array->d_array[i] = D_array->d_array[i + 1];
144     }
145     D_array->size--;
146     return D_array;
147 }
```

dynamic_array_delete_front 함수에서는 DynamicArray 구조체를 입력받고 배열 전체를 한칸 앞으로 당깁니다. 그리고 size 를 1 줄이고 DynamicArray 구조체를 반환합니다. 만약 삭제할 요소가 없다면 그대로 반환합니다.

```
149 // deleting an element(back)
150 struct DynamicArray* dynamic_array_delete_back(DynamicArray* D_array) {
151     if (D_array->size == 0) {
152         return D_array; // 삭제할 요소가 없는 경우 그대로 반환
153     }
154
155     D_array->size--;
156     return D_array;
157 }
```

dynamic_array_delete_back 함수에서는 DynamicArray 구조체를 입력받고 배열의 size 를 한칸 줄이고 DynamicArray 구조체를 반환합니다. 만약 삭제할 요소가 없다면 그대로 반환합니다.

```

159 // deleting an element(specific)
160 struct DynamicArray* dynamic_array_delete_specific(DynamicArray* D_array, int delete_index) {
161     if (D_array->size == 0) {
162         return D_array; // 삭제할 요소가 없는 경우 그대로 반환
163     }
164     if (delete_index == 0) {
165         D_array = dynamic_array_delete_front(D_array);
166         return D_array;
167     }
168     if (delete_index == D_array->size - 1) {
169         D_array = dynamic_array_delete_back(D_array);
170         return D_array;
171     }
172     else if (delete_index < 0 || delete_index >= D_array->size) {
173         printf("index out!!\n");
174         return D_array;
175     }
176     else {
177         for (int i = delete_index; i < D_array->size; i++) {
178             D_array->d_array[i] = D_array->d_array[i + 1];
179         }
180         D_array->size--;
181         return D_array;
182     }
183 }

```

dynamic_array_delete_specific 함수에서는 DynamicArray 구조체와 delete_index 를 입력받습니다. 삭제할 요소가 없는 경우에는 그대로 반환하고, 만약 삭제할 인덱스가 0 이라면 dynamic_array_delete_front 함수를 호출하여 0 번 element 를 삭제후 DynamicArray 구조체를 반환합니다. 만약 삭제할 인덱스가 배열의 맨 마지막이라면 dynamic_array_delete_back 함수를 호출하여 맨 마지막 element 를 삭제후 DynamicArray 구조체를 반환합니다. 인덱스가 0 이나 배열의 맨 마지막이 아닌 경우 delete_index 자리 + 1 번 element 부터 맨 끝까지를 한칸 앞으로 당기고, size 를 1 줄이고 반환합니다.

```

185 // access an element at a specific index
186 int dynamic_array_access(DynamicArray* D_array, int access_index) {
187     if (access_index < 0 || access_index >= D_array->size) {
188         printf("index out!!\n");
189         return 0;
190     }
191     int* return_element = D_array->d_array + (access_index);
192     return*return_element;
193 }

```

dynamic_array_access 함수에서는 DynamicArray 구조체와 access_index 를 입력받고, DynamicArray 의 배열에서 d_array + (access_index) 로 접근하여 return element 를 반환합니다. access_index 가 배열을 벗어날 경우 error 메시지를 출력합니다.

```

195 // dynamic array 출력 함수
196 void dynamic_array_print(DynamicArray* D_array) {
197     for (int i = 0; i < D_array->size; i++)
198     {
199         printf("%d ", D_array->d_array[i]);
200     }
201     printf("\nSize : %d, Capacity: %d\n\n", D_array->size, D_array->capacity);
202 }

```

DynamicArray 구조체를 출력하는 함수로 배열이 내용과 size, capacity 를 출력합니다. 테스트 목적으로 사용합니다.

```

205 typedef struct Singly_linked_lists {
206     struct Singly_linked_lists* next; // 다음 노드를 가리키는 포인터
207     int data; // 노드의 데이터
208 } Singly_linked_lists;
209

```

Singly_linked_lists 구조체를 정의합니다. data 와 다음 노드를 가리키는 포인터로 구성되어 있습니다.

```

210 // linked_list initialization (empty list)
211 struct Singly_linked_lists* linked_list_init() {
212     // 링크드리스트는 개수를 정하는게 아니라 노드 하나를 생성하고 반환하기
213     Singly_linked_lists* linked_list = (struct Singly_linked_lists*)malloc(sizeof(struct Singly_linked_lists));
214     linked_list->next = NULL;
215     linked_list->data = 0;
216     return linked_list;
217 }

```

linked_list_init 함수에서 링크드리스트는 개수를 정하는 것이 아닌, 노드 하나를 생성하고 반환하도록 합니다. 메모리를 할당받은 후, 다음 노드는 NULL 이고, data 는 0 으로 초기화 합니다.

```

219 // inserting an element(front)
220 struct Singly_linked_lists* linked_list_insert_front(Singly_linked_lists* linked_list, int insert_element) {
221     Singly_linked_lists* linked_list_front = linked_list_init();
222     linked_list_front->data = insert_element;
223     //앞에 연결
224     linked_list_front->next = linked_list->next;
225     return linked_list_front;
226 }

```

linked_list_insert_front 함수에서는 linked_list_init 함수로 앞에 넣을 노드인 linked_list_front 를 생성하여 data 를 넣고, linked_list_front 의 next 포인터가 linked_list 를 가리키도록 합니다. 그 뒤에 linked_list_front 를 반환합니다.

```

228 // inserting an element(back)
229 struct Singly_linked_lists* linked_list_insert_back(Singly_linked_lists* linked_list, int insert_element) {
230     Singly_linked_lists* linked_list_back = linked_list_init();
231     linked_list_back->data = insert_element;
232     //뒤에 연결
233     Singly_linked_lists* start = linked_list; // 처음 노드를 저장하고 반환
234     while (linked_list->next != NULL) {
235         linked_list = linked_list->next;
236     }
237     linked_list->next = linked_list_back;
238     return start;
239 }

```

linked_list_insert_back 함수에서는 뒤에 넣을 노드인 linked_list_back 을 생성하고 data 를 넣습니다. linked_list의 처음 노드를 start에 저장하고 while문을 통해 처음부터 마지막 노드까지 접근합니다. 마지막 노드의 next 포인터가 linked_list_back 을 가리키도록 하고, start 를 반환합니다.

```

241 // insert specific 함수에서 마지막 삽입의 경우 insert back 함수를 호출하기 위해 연결리스트 길이를 반환
242 int linked_list_size(Singly_linked_lists* linked_list) {
243     int size = 0;
244     Singly_linked_lists* l = linked_list;
245     while (l != NULL) {
246         size++;
247         l = l->next;
248     }
249     return size;
250 }

```

linked_list_size 함수에서는 Singly_linked_list 구조체의 size 를 노드의 맨 처음부터 끝까지 접근하여 count 하고 반환합니다. 이 함수는 linked list 의 특정 위치에 삽입하거나 삭제할때 사용됩니다.

```

252 // inserting an element(specific)
253 struct Singly_linked_lists* linked_list_insert_specific(Singly_linked_lists* linked_list, int insert_element, int insert_index) {
254     if (insert_index == 0) {
255         linked_list = linked_list_insert_front(linked_list, insert_element);
256         return linked_list;
257     }
258     if (insert_index == linked_list_size(linked_list)) { // 마지막 삽입인 경우 뒤에 삽입하는 함수 호출
259         linked_list = linked_list_insert_back(linked_list, insert_element);
260         return linked_list;
261     }
262     Singly_linked_lists* linked_list_new = linked_list_init();
263     linked_list_new->data = insert_element;
264
265     Singly_linked_lists* start = linked_list; // 처음 노드를 저장하고 반환
266     // insert index - 1 까지 접근
267     int count = 0;
268     while (count != insert_index - 1) {
269         count++;
270         linked_list = linked_list->next;
271     }
272     linked_list->next = linked_list_new;
273     linked_list->next = linked_list_new;
274     return start;
275 }

```

linked_list_insert_specific 함수는 Singly_linked_list 구조체와 insert_element, insert_index 를 입력받습니다. 먼저 linked_list_new 을 생성하고 data 에 insert element 를 넣습니다. 만약 insert_index 가 0 이라면 linked_list_insert_front 함수를 호출하여 맨 앞에 삽입후 반환합니다. 만약 insert_index 가 linked_list_size 함수를 호출하여 측정한 길이의 맨 끝이라면

linked_list_insert_back 함수를 호출하여 삽입후 반환합니다. Insert_index 가 처음이나 맨 끝이 아니라면 start 에 처음 노드를 저장하고, linked list 의 처음부터 insert index - 1 까지 접근한 후, linked_list_new 의 next 포인터가 현재 위치 노드의 next 를 가리키도록 하고, 현재위치의 next 포인터가 linked_list_new 를 가리키도록 하고 start 를 반환합니다.

```
277 // deleting an element(front)
278 struct Singly_linked_lists* linked_list_delete_front(Singly_linked_lists* linked_list) {
279     if (linked_list->next == NULL || linked_list == NULL) {
280         return NULL;
281     }
282     Singly_linked_lists* del_node = linked_list;
283     linked_list = linked_list->next;
284     free(del_node);
285     return linked_list;
286 }
```

linked_list_delete_front 함수에서는 Singly_linked_list 구조체를 입력받고 맨 앞을 del_node 가 맨 앞을 가리키도록 합니다. linked_list 를 linked_list 의 next 가 가리키는 대상으로 바꾼 뒤에 del_node 의 메모리를 해제합니다. 그 뒤 linked_list 를 반환합니다.

```
288 // deleting an element(back)
289 struct Singly_linked_lists* linked_list_delete_back(Singly_linked_lists* linked_list) {
290     if (linked_list->next == NULL || linked_list == NULL) {
291         return NULL;
292     }
293     Singly_linked_lists* start = linked_list;
294     while (linked_list->next->next != NULL) {
295         linked_list = linked_list->next;
296     }
297     Singly_linked_lists* del_node = linked_list->next;
298     linked_list->next = NULL;
299     free(del_node);
300     return start;
301 }
```

linked_list_delete_back 함수에서는 Singly_linked_list 구조체를 입력받고 start 가 linked_list 를 가리키도록 합니다. 처음부터 linked_list 의 next->next 가 NULL 을 가리킬때까지 접근하고, del_node 가 linked_list 의 next 가 가리키는 대상이 되도록 합니다. del_node 의 메모리를 해제하고 start 를 반환합니다.

```

303 // deleting an element(specific)
304 struct Singly_linked_lists* linked_list_delete_specific(Singly_linked_lists* linked_list, int delete_index) {
305     if (linked_list->next == NULL || linked_list == NULL) {
306         return NULL;
307     }
308     if (delete_index == 0) {
309         linked_list = linked_list_delete_front(linked_list);
310         return linked_list;
311     }
312     else if (delete_index == linked_list_size(linked_list)) {
313         linked_list = linked_list_delete_back(linked_list);
314         return linked_list;
315     }
316     else {
317         Singly_linked_lists* start = linked_list;
318         int count = 0;
319         while (count + 1 != delete_index) {
320             linked_list = linked_list->next;
321             count++;
322         }
323         Singly_linked_lists* del_node = linked_list->next;
324         linked_list->next = linked_list->next->next;
325
326         free(del_node);
327         return start;
328     }
329 }
330

```

linked_list_delete_specific 함수에서는 Singly_linked_list 구조체와 delete_index 를 입력받고, delete_index 가 0 이라면 linked_list_delete_front 함수를 호출하여 맨 앞 노드를 지우고 반환합니다. delete_index 가 linked list 의 맨 끝이라면 linked_list_delete_back 함수를 호출하여 맨 뒤 노드를 지우고 반환합니다.

delete_index 가 0 이나 맨 끝이 아니라면 linked_list 의 시작을 start 포인터가 가리키도록 하고, delete_index 의 한 칸 이전까지 접근합니다. del_node 가 현재 위치의 next 포인터가 가리키는 대상이 되도록 하고, 현재 위치의 next 포인터가 가리키는 대상은 그 다음 다음 노드가 되도록 합니다. del_node 의 메모리를 해제하고, start 를 반환합니다.

```

331 // accessing the element at the head
332 int linked_list_access(Singly_linked_lists* linked_list, int access_index) {
333     int count = 0;
334     while (count < access_index) {
335         count++;
336         if (linked_list->next == NULL) {
337             break;
338         }
339         linked_list = linked_list->next;
340     }
341
342     if (linked_list == NULL) { // Null경우 0 반환하기
343         printf("error");
344         return 0;
345     }
346     else {
347         return linked_list->data;
348     }
349 }
350

```

linked_list_access 함수에서는 Singly_linked_list 구조체와 access_index 를 입력받고 처음부터 access_index 까지 접근합니다. 만약 index 범위를 넘어간다면 break 문으로 반복문을 빠져나와서 마지막 대상을 가리키도록 하여 error 가 없게 합니다.

Linked list 가 Null 인 경우에는 error 메시지를 출력하고 0 을 반환합니다. Access_index 까지 접근하였다면 노드의 data 를 반환합니다.

```
351 // linked list 출력함수
352 void linked_list_print(Singly_linked_lists* linked_list) {
353     int count = 0;
354     while (linked_list != NULL) {
355         printf("%d ", linked_list->data);
356         count++;
357         linked_list = linked_list->next;
358     }
359     printf("\nsize : %d\n\n", count);
360 }
361
```

Singly_linked_list 구조체를 출력하는 함수로 처음부터 접근하여 data 를 출력하고 size 도 count 합니다. 테스트 목적으로 사용합니다.

```
366 //DynamicArray insert test 함수, num은 data의 개수
367 void test_insert_DynamicArray(int num) {
368     DynamicArray* D1 = dynamic_array_init(2, 2); //capacity 2, size 2의 동적배열(size가 0이면 mod연산에서 문제가 생길수 있으므로)
369     StartTimer();
370     for (int i = 0; i < num; i++) {
371         int random_index = rand() % D1->size;
372         dynamic_array_insert_specific(D1, 1, random_index);
373     }
374     //dynamic_array_print(D1);
375     printf("Dynamic array insert\n");
376     double elapsedTime = EndTimer(); // 경과 시간 계산 및 반환
377     printf("Elapsed Time: %.20f seconds\n", elapsedTime * 0.000001);
378     free(D1->d_array);
379     free(D1);
380 }
381
```

동적 배열의 삽입 연산의 시간을 측정하는 함수입니다. 랜덤 인덱스를 위한 mod 연산에 문제가 없도록 처음 init 하는 DynamicArray 의 capacity, size 는 기본값을 2 로 하고, 매개변수로 입력받은 num 만큼 랜덤한 인덱스에 삽입 연산을 반복합니다. 경과시간을 출력하고, 메모리 누수가 없게 init 했던 동적 배열의 메모리를 해제합니다.

```

388 //DynamicArray delete test 함수
389 void test_delete_DynamicArray(int num) {
390     DynamicArray* D1 = dynamic_array_init(num, num);
391
392     StartTimer();
393
394     for (int i = 0; i < num; i++) {
395         int random_index = rand() % D1->size;
396         dynamic_array_delete_specific(D1, random_index);
397     }
398     //dynamic_array_print(D1);
399
400     printf("Dynamic array delete\n");
401
402     double elapsedTime = EndTimer(); // 경과 시간 계산 및 반환
403     printf("Elapsed Time: %.20f seconds\n", elapsedTime * 0.000001);
404
405     free(D1->d_array);
406     free(D1);
407 }

```

동적 배열의 삭제 연산의 시간을 측정하는 함수입니다. 매개변수로 입력받은 num 만큼의 크기로 동적배열의 size, capacity 를 init 하고, 랜덤한 위치의 요소를 num 번 반복하여 삭제 연산을 합니다. 경과시간을 출력하고, 메모리 누수가 없게 init 했던 동적 배열의 메모리를 해제합니다.

```

409 //DynamicArray access test 함수
410 void test_access_DynamicArray(int num) {
411     DynamicArray* D1 = dynamic_array_init(num, num);
412
413     StartTimer();
414
415     for (int i = 0; i < num; i++) {
416         int random_index = rand() % D1->size;
417         int access = dynamic_array_access(D1, random_index);
418     }
419     //dynamic_array_print(D1);
420
421     printf("Dynamic array access\n");
422
423     double elapsedTime = EndTimer(); // 경과 시간 계산 및 반환
424     printf("Elapsed Time: %.20f seconds\n", elapsedTime * 0.000001);
425
426     free(D1->d_array);
427     free(D1);
428 }
429

```

동적 배열의 access 연산의 시간을 측정하는 함수입니다. 매개변수로 입력받은 num 만큼의 크기로 동적배열의 size, capacity 를 init 하고, 랜덤한 위치의 요소 값을 반환합니다. 경과시간을 출력하고, 메모리 누수가 없게 init 했던 동적 배열의 메모리를 해제합니다.

```

430 // LinkedList insert test함수
431 void test_insert_LinkedList(int num) {
432     Singly_linked_lists* L1 = linked_list_init();
433
434     StartTimer();
435
436     for (int i = 0; i < num; i++) {
437         int random_index = rand() % linked_list_size(L1);
438         linked_list_insert_specific(L1, 1, random_index);
439     }
440     //linked_list_print(L1);
441
442     printf("Linked list insert\n");
443
444     double elapsedTime = EndTimer(); // 경과 시간 계산 및 반환
445     printf("Elapsed Time: %.20f seconds\n", elapsedTime * 0.000001);
446
447     free(L1);
448 }

```

링크드리스트의 삽입 연산의 시간을 측정하는 함수입니다. 처음에 linked_list_init 함수로 한 개의 노드를 생성하고, 랜덤한 인덱스에 삽입 연산을 매개변수로 입력받은 num 회 만큼 반복합니다. 경과시간을 출력하고 init 했던 링크드리스트의 메모리를 해제합니다.

```

450 // LinkedList delete test함수
451 void test_delete_LinkedList(int num) {
452     Singly_linked_lists* L1 = linked_list_init();
453     for (int i = 0; i < num; i++) {
454         L1 = linked_list_insert_front(L1, 1); //길이 num의 linked list를 만든다
455     }
456
457     StartTimer();
458
459     for (int i = 0; i < num; i++) {
460         int random_index = rand() % linked_list_size(L1);
461         linked_list_delete_specific(L1, random_index);
462     }
463     //linked_list_print(L1);
464
465     printf("Linked list delete\n");
466
467     double elapsedTime = EndTimer(); // 경과 시간 계산 및 반환
468     printf("Elapsed Time: %.20f seconds\n", elapsedTime * 0.000001);
469
470     free(L1);
471 }

```

링크드리스트의 삭제 연산의 시간을 측정하는 함수입니다. 처음에 linked_list_init 함수로 한 개의 노드를 생성하고, 삽입 연산을 매개변수로 입력받은 num 회 만큼 반복하여 길이 num + 1 의 링크드리스트를 생성합니다.

그 뒤로 매개변수로 입력받은 num 회 만큼 반복하여 랜덤한 인덱스에 삭제 연산을 합니다.
경과시간을 출력하고 init 했던 링크드리스트의 메모리를 해제합니다.

```
473 // LinkedList access test함수
474 void test_access_LinkedList(int num) {
475     Singly_linked_lists* L1 = linked_list_init();
476     for (int i = 0; i < num; i++) {
477         L1 = linked_list_insert_front(L1, 1); //길이 num의 linked list를 만든다
478     }
479
480     StartTimer();
481
482     for (int i = 0; i < num; i++) {
483         int random_index = rand() % linked_list_size(L1);
484         int access = linked_list_access(L1, random_index);
485         //printf("access : %d",access);
486     }
487     //linked_list_print(L1);
488
489     printf("Linked list access\n");
490
491     double elapsedTime = EndTimer(); // 경과 시간 계산 및 반환
492     printf("Elapsed Time: %.20f seconds\n", elapsedTime * 0.000001);
493
494     free(L1);
495 }
```

링크드리스트의 access 연산의 시간을 측정하는 함수입니다. 처음에 linked_list_init 함수로 한 개의 노드를 생성하고, 삽입 연산을 매개변수로 입력받은 num 회 만큼 반복하여 길이 num + 1 의 링크드리스트를 생성합니다.

그 뒤로 매개변수로 입력받은 num 회 만큼 반복하여 랜덤한 인덱스에 access 연산을 합니다.
경과시간을 출력하고 init 했던 링크드리스트의 메모리를 해제합니다

```
497 // test insert, delete, access
498 void test(int size) { // vary the number of elements in the data structure
499     printf("\ndata size is %d\n", size);
500     test_insert_DynamicArray(size);
501     test_delete_DynamicArray(size);
502     test_access_DynamicArray(size);
503
504     test_insert_LinkedList(size);
505     test_delete_LinkedList(size);
506     test_access_LinkedList(size);
507 }
```

test 함수에서는 매개변수로 element 의 개수를 입력받도록 하고, 동적배열과 링크드리스트의 insert, delete, access 시간측정 함수를 호출합니다.

```

509 int main() {
510     srand(time(NULL)); // 난수 초기화
511
512     test(10);
513     test(100);
514     test(1000);
515     test(10000);
516
517     return 0;
518 }

```

main 함수에서는 난수를 초기화하고, test 함수에 10, 100, 1000, 10000 을 매개변수로 넣고 실행시간을 측정하도록 합니다.

1-2 : Time Complexity Analysis

```

data size is 10
Dynamic array insert
Elapsed Time: 0.000033800000000000 seconds
Dynamic array delete
Elapsed Time: 0.000031700000000000 seconds
Dynamic array access
Elapsed Time: 0.000040900000000000 seconds
Linked list insert
Elapsed Time: 0.000031600000000000 seconds
Linked list delete
Elapsed Time: 0.000029700000000000 seconds
Linked list access
Elapsed Time: 0.000042999999999999 seconds

```

```

data size is 100
Dynamic array insert
Elapsed Time: 0.000055400000000000 seconds
Dynamic array delete
Elapsed Time: 0.000036900000000000 seconds
Dynamic array access
Elapsed Time: 0.000092200000000000 seconds
Linked list insert
Elapsed Time: 0.000105699999999998 seconds
Linked list delete
Elapsed Time: 0.000036799999999999 seconds
Linked list access
Elapsed Time: 0.000038800000000000 seconds

```

```

data size is 1000
Dynamic array insert
Elapsed Time: 0.000659199999999998 seconds
Dynamic array delete
Elapsed Time: 0.000632199999999998 seconds
Dynamic array access
Elapsed Time: 0.000153800000000000 seconds
Linked list insert
Elapsed Time: 0.000445799999999999 seconds
Linked list delete
Elapsed Time: 0.000113000000000000 seconds
Linked list access
Elapsed Time: 0.000298299999999999 seconds

```

```

data size is 10000
Dynamic array insert
Elapsed Time: 0.050519099999999734 seconds
Dynamic array delete
Elapsed Time: 0.0444051999999999190 seconds
Dynamic array access
Elapsed Time: 0.000472800000000000 seconds
Linked list insert
Elapsed Time: 0.0019751999999999987 seconds
Linked list delete
Elapsed Time: 0.000606600000000000 seconds
Linked list access
Elapsed Time: 0.0005295999999999997 seconds

```

연산 횟수를 바꾸면서 실행 결과를 출력했고, The number of elements in the data structure 가 커질수록 실행시간이 증가하는 것을 확인할 수 있다. 숫자가 작을수록 실행시간이 지나치게 작아져서 측정이 되지 않는 문제가 있었으나 시간을 측정하는데 QueryPerformanceCounter() 함수를 사용하여 해결할 수 있었다.

Dynamic array 에서 insert 연산은 back 의 경우 시간복잡도가 $O(1)$ 이지만, front 의 경우 모든 요소를 한 칸씩 뒤로 이동해야 하므로 시간복잡도가 $O(n)$ 이고, specific 의 경우 삽입 지점 이후의 모든 요소를 한 칸씩 이동해야 하므로 시간복잡도가 $O(n)$ 이다.

Delete 연산은 back 의 경우 시간복잡도가 $O(1)$ 이고, front 의 경우 모든 요소를 한 칸씩 앞으로 옮겨야 하기에 시간복잡도가 $O(n)$ 이고, specific 의 경우 삭제 지점 이후의 모든 요소를 한 칸씩 앞으로 옮기므로 시간복잡도가 $O(n)$ 이다.

Access 연산은 특정 인덱스의 요소에 접근하므로 $O(1)$ 이다.

Linked list 에서 insert 연산은 front 와 back 에 삽입하는 경우 시간복잡도가 $O(1)$ 이며, specific 의 경우 처음부터 삽입하려는 위치를 찾아가므로 시간복잡도가 $O(n)$ 이다.

Delete 연산은 front, back 의 경우 시간복잡도가 $O(1)$ 이며, specific 의 경우 처음부터 삭제 위치를 찾아가므로 시간복잡도가 $O(n)$ 이다.

Access 연산은 처음부터 원하는 위치까지 순차적으로 탐색해야 하므로 시간복잡도가 $O(n)$ 이다.

위의 코드의 실행 결과를 통해서 이 경향성을 어느정도 확인할 수가 있는데, 시간복잡도가 $O(1)$ 의 Dynamic array 의 access 연산은 data size 가 커져도 실행시간 증가폭이 작은 것을 확인할 수 있고, 랜덤한 인덱스를 이용했기에 시간복잡도가 평균적으로 $O(n)$ 인 insert, back 연산의 경우 실행 속도가 $O(1)$ 인 access 연산에 비해서 급격히 증가하는 것을 확인할 수 있다.

랜덤한 인덱스를 사용하도록 했으므로 평균적으로 시간복잡도가 $O(n)$ 인 linked list 의 insert, delete, access 연산은 data size 가 증가할수록 실행시간이 증가하는 것을 확인할 수 있다.

Dynamic array 의 경우 데이터 크기가 증가할수록 실행시간이 linked list 에 비해 급격하게 증가하는 것을 확인할 수 있는데, 이것은 배열의 크기를 조절하는 과정으로 인한 것으로 생각된다.

Linked list 는 포인터의 이동만으로 insert, delete 연산이 이루어지기 때문에 실행시간 증가폭이 dynamic array 에 비해 작은 것을 실행 결과를 통해서 확인할 수 있다.

데이터의 크기가 작고 insert, delete 연산을 자주 해야 하는 경우에는 linked list 를 사용하는 것이 좋다는 것을 알 수 있고, 데이터의 access 속도가 중요한 경우 access 의 시간복잡도가 $O(1)$ 인 dynamic array 를 사용하는 것이 좋다는 것을 알 수 있다.

1-3 : Space Complexity Analysis

Dynamic array 에서 insert 연산은 front 의 경우 배열의 크기를 조정하는 경우 새 메모리 공간이 필요하다. Insert 의 경우 capacity 를 넘어갈 때 동적배열의 크기를 2 배로 다시 할당받고 있으므로

공간복잡도는 $O(n)$ 이다. Delete 의 경우 메모리 공간이 추가로 필요하지 않고 $O(1)$ 이다. Access 연산의 경우 인덱스를 통해서 접근하며 $O(1)$ 이다.

Linked list 의 경우 insert 연산은 새 노드를 위한 메모리 할당만 필요하므로 $O(1)$ 이고, Delete 연산은 삭제하는 요소의 메모리를 해제하면 되므로 $O(1)$ 이다. Access 의 경우 순차적으로 노드를 탐색하므로 $O(1)$ 이다. 그렇지만 각 노드에 포인터를 저장하고 있으므로 dynamic array 에 비해 연산 오버헤드가 있다.

Linked list 의 경우 각 노드마다 data 와 포인터를 저장해야 하므로 메모리 공간의 소비가 더 많을 수 있다. 그러나 Dynamic array 는 배열의 크기를 조절하는 과정에서 추가로 메모리를 할당받게 된다. 큰 데이터를 삽입, 삭제하는 경우에는 linked list 에 비해 메모리를 많이 사용하게 된다.